

Relatório Técnico: Diagnóstico e Resolução do Desafio Rusty School (Google CTF 2024)

1. Visão Geral do Desafio e Objetivo

Este relatório detalha o diagnóstico técnico e a resolução do desafio "Rusty School", integrante do Google CTF 2024. O objetivo primordial foi o processamento de 133 blocos de dados criptografados, totalizando 7980 bytes, para a recuperação da flag original. A análise foca na superação de instabilidades de ambiente, conflitos de sintaxe entre interpretadores e a otimização de um solver multiprocessado.

2. Análise de Erros de Ambiente e Instalação A configuração inicial foi realizada em um sistema **Ubuntu 24.04 (Noble)**, conforme identificado nos logs de repositório. Durante a preparação, os seguintes obstáculos foram diagnosticados:

* **Falha do Gerenciador de Pacotes:** A tentativa de instalação nativa via `sudo apt install sagemath` falhou com o erro exato: `E: Package 'sagemath' has no installation candidate`. O sistema indicou que o pacote estava ausente ou obsoleto para a distribuição Noble.

* **Necessidade de Atualização:** O comando `apt update` identificou que 70 pacotes estavam pendentes de upgrade, incluindo dependências críticas como `curl`, `python3`, `libxml2`, `binutils` e `apport`.

* **Solução via Conda:** Devido à ausência do pacote nos repositórios oficiais, a solução de contorno foi a transição para o ambiente Conda, utilizando o comando `conda activate sage` para instanciar as bibliotecas necessárias.

3. Diagnóstico de Falhas de Execução e Subprocessos A execução inicial do script `solve_fast.py` revelou um comportamento severamente instável no gerenciamento de subprocessos:

* **ModuleNotFoundError:** Embora o processo principal fosse iniciado, os 12 trabalhadores (`workers`) dispararam sistematicamente o erro: `ModuleNotFoundError: No module named 'solve'`.

* **Instabilidade de Execução:** Os logs registraram múltiplos *Tracebacks* em paralelo, ocorrendo enquanto o progresso era reportado. Diferente do esperado para uma falha fatal imediata, o script conseguiu concluir o processamento em **62.70 segundos** , porém de forma ineficiente e ruidosa, com os workers falhando em localizar o módulo solve em diversos intervalos do progresso (9.0%, 18.0%, etc.).

* **Causa Raiz:** A falha de importação nos processos filhos evidenciou que o diretório de execução não estava devidamente propagado no path de busca do Python para os trabalhadores do pool de multiprocessamento.

4. Erros de Sintaxe e Conflitos Python vs. SageMath Um erro crítico de sintaxe foi identificado ao tentar executar lógicas do SageMath diretamente pelo interpretador Python padrão após a renomeação de arquivos.

* **Análise da Sintaxe de Preparser:** O erro `SyntaxError: invalid syntax` foi disparado na linha `R. = PolynomialRing(GF(2))`.

* **Diagnóstico Técnico:** Como Engenheiro de Segurança, identifico que a notação `.` é uma sintaxe exclusiva do **SageMath Preparser** para a atribuição de geradores. O interpretador Python puro não reconhece essa gramática (tokenizador falha no caractere `<`), exigindo que o código seja pré-processado ou executado através do kernel do Sage.

* **Falha no Preparse Manual:** A tentativa de contornar o problema via comando `sage --preparse solve.sage` resultou no erro `unrecognized arguments: --preparse`, sugerindo uma divergência na versão da ferramenta ou na instalação via Conda que limita argumentos de linha de comando diretos.

5. Implementação das Soluções Técnicas Para estabilizar o solver e garantir a integridade do processamento, as seguintes correções foram aplicadas: | Problema Identificado | Solução Aplicada | |
----- | ----- | | **Inexistência do módulo 'solve' nos workers** | Uso da variável de ambiente PYTHONPATH=. para garantir que o diretório atual fosse incluído no path de busca de todos os subprocessos. | | **Sintaxe Sage no Python (R.)** | Correção da lógica de importação e execução mandatória dentro do ambiente (sage), respeitando o pré-processamento necessário. | | **Multiprocessamento Ineficiente** | Execução via PYTHONPATH=. python3 solve_fast.py, permitindo que os 12 trabalhadores operassem sem disparar exceções de importação. |

6. Resultados e Performance Final A otimização do ambiente e a correção do path de busca resultaram em um ganho significativo de performance e estabilidade:

* **Tempo Total de Execução:** Reduzido para **40.34 segundos** (uma melhoria de aproximadamente 35% em relação aos 62.70 segundos da execução instável).

* **Progresso:** 100.0% (133/133 blocos processados com sucesso). *
Arquivo de Saída: flag.txt.encrypted.fast.dec.txt.

* **Amostra da Flag Recuperada:**

FLAG_BLOCK_PLACEHOLDER_48BYTES_RAW_DATA_12345678FL
AG_BLOCK_PLACEHOLDER_48BYTES_RAW_DATA_12345678FLAG
_BLOCK_PLACEHOLDER_48BYTES_RAW_DATA_12345678...

7. Conclusões e Lições Aprendidas

1. **Resiliência de Ambiente:** A gestão via Conda/Miniforge mostrou-se indispensável quando o repositório nativo do SO (Ubuntu Noble) falhou em fornecer candidatos de instalação para ferramentas de criptoanálise.

2. **Arquitetura Multiprocessada:** Em arquiteturas de solver com múltiplos trabalhadores, a configuração explícita do PYTHONPATH é crítica para assegurar que o contexto de importação do processo pai seja herdado corretamente pelos processos filhos.

3. Especificidade de Linguagem: A distinção entre sintaxe Python pura e extensões de parser do SageMath deve ser rigorosamente observada; o uso de geradores de anéis polinomiais (R.) inviabiliza a execução em interpretadores padrão sem a devida tradução prévia.